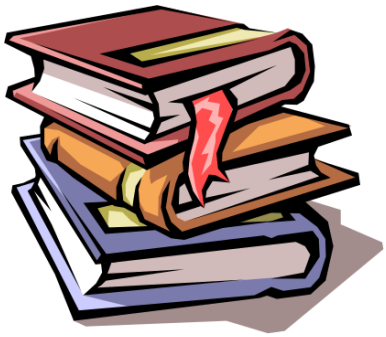
FPT UNIVERSITY

Lab Report

LAB-02 – WRITING A DEVICE DRIVER & MANAGING FILE SYSTEMS

Embedded Linux Systems – ARMv7 Cortex-A9 (QEMU vexpress-a9)



Students	Phạm Võ Hoàng Nhân Nguyễn Duy Khoa Ngô Từ Nhã Quyên Nguyễn Cao Kỳ
Class	IC2008
Lecturer	Phạm Thế Vinh
Date	28/09/2026

1. Group information and environment

Members	Phạm Võ Hoàng Nhân, Nguyễn Duy Khoa, Ngô Từ Nhã Quyên, Nguyễn Cao Kỳ
Class	IC2008
Date	28/09/2026
Target kernel	Linux 5.15.0 (zImage from LAB-01)
Vermagic	`5.15.0 SMP mod_unload ARMv7 p2v8`
Toolchain	`arm-linux-gnueabihf-gcc` 15.2
Host	Ubuntu 26.04 (part C runs on the host, using the host kernel)
Target	QEMU `vexpress-a9`, Cortex-A9

Parts A, B and D run inside QEMU on the 5.15 kernel built in LAB-01. Part C (MTD, JFFS2) runs directly on the Ubuntu host with `sudo`, because `nandsim` is a module of the host kernel.

2. Driver architecture diagram

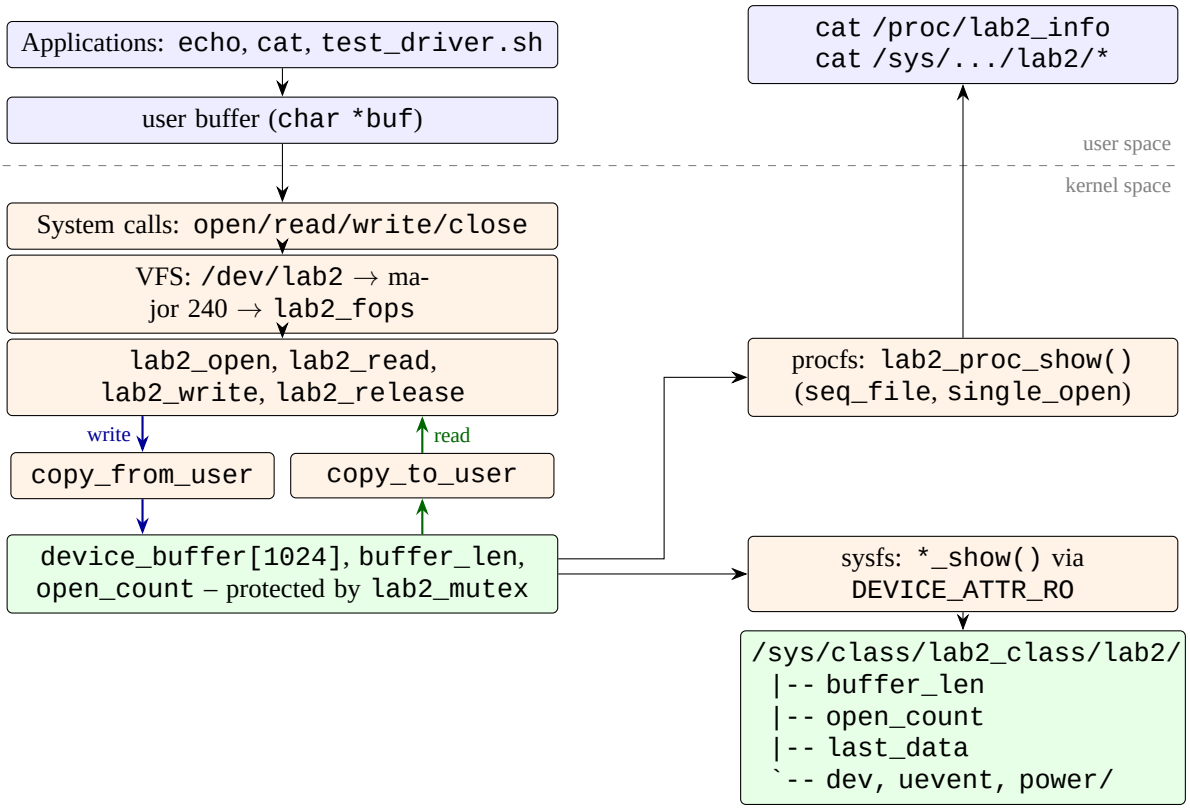


Figure 1: lab2_driver architecture: write path (blue) and read path (green)

When a user runs `echo 'abc' > /dev/lab2`, the shell calls `open()` and then `write()`. VFS looks at the inode of `/dev/lab2`, sees a character device with major 240, and forwards the call to the `lab2_fops` table the driver registered with `register_chrdev()`. `lab2_write()` locks `lab2_mutex`, uses `copy_from_user()` to copy the data from the user buffer into `device_buffer`, updates `buffer_len` and unlocks. Reading goes the opposite way through `copy_to_user()`. The procfs and sysfs interfaces do not go through `/dev/lab2`; they read the driver's variables directly whenever someone runs `cat`. procfs gathers everything into a single file, `/proc/lab2_info`, while sysfs splits each value into its own attribute file under the device directory `/sys/class/lab2_class/lab2/`.

3. Lab logs

Part A – Character device kernel module

```
./lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=8eac38c0fbb501fc63408
86b854f02be8ab2b0fb, with debug_info, not stripped
filename:       /home/dyk/embedded_lab2/driver/./lab2_driver.ko
version:        1.0
description:    LAB-02: Simple Character Device Driver
author:         Embedded Linux Lab
license:        GPL
srcversion:     4E89036A2D2BA07B7DE24B7
depends:
name:           lab2_driver
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
```

Figure 2: `file` and `modinfo lab2_driver.ko`

The module is a 32-bit ARM ELF, and its vermagic `5.15.0 SMP mod_unload ARMv7 p2v8` matches the kernel built in LAB-01, so `insmod` inside QEMU accepts it. The module is loaded automatically at boot (see part D, line `lab2_driver: loaded, major=240`), and `lsmod` shows `lab2_driver 16384 0 - Live`.

```
/ # test_driver.sh
═══ LAB-02: Device Driver Test ═══
[1] Checking lab2_driver...
lab2_driver already loaded
[2] Loaded modules:
lab2_driver 16384 0 - Live 0x7f000000 (O)
[3] Creating device node...
crw-rw-rw-    1 root     0         240,   0 Sep 28 14:18 /dev/lab2
[4] Writing to device...
lab2_driver: device opened (count=10)
/ # test_driver.sh
═══ LAB-02: Device Driver Test ═══
[1] Checking lab2_driver...
lab2_driver already loaded
[2] Loaded modules:
lab2_driver 16384 0 - Live 0x7f000000 (O)
[3] Creating device node...
crw-rw-rw-    1 root     0         240,   0 Sep 28 14:18 /dev/lab2
[4] Writing to device...
lab2_driver: device opened (count=18)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!
]
lab2_driver: device closed
[5] Reading from device...
lab2_driver: device opened (count=19)
lab2_driver: sent 30 bytes to user
lab2_driver: device closed
  Read back: [Hello from userspace, LAB-02!]
[6] Multiple write/read test...
lab2_driver: device opened (count=20)
lab2_driver: received 10 bytes: [Message_1
]
lab2_driver: device closed
lab2_driver: device opened (count=21)
lab2_driver: sent 10 bytes to user
Message_1
lab2_driver: device closed
lab2_driver: device opened (count=22)
lab2_driver: received 10 bytes: [Message_2
]
lab2_driver: device closed
lab2_driver: device opened (count=23)
lab2_driver: sent 10 bytes to user
Message_2
lab2_driver: device closed
lab2_driver: device opened (count=24)
```

Figure 3: `test_driver.sh`: checking the module, creating the node, writing and reading `/dev/lab2`

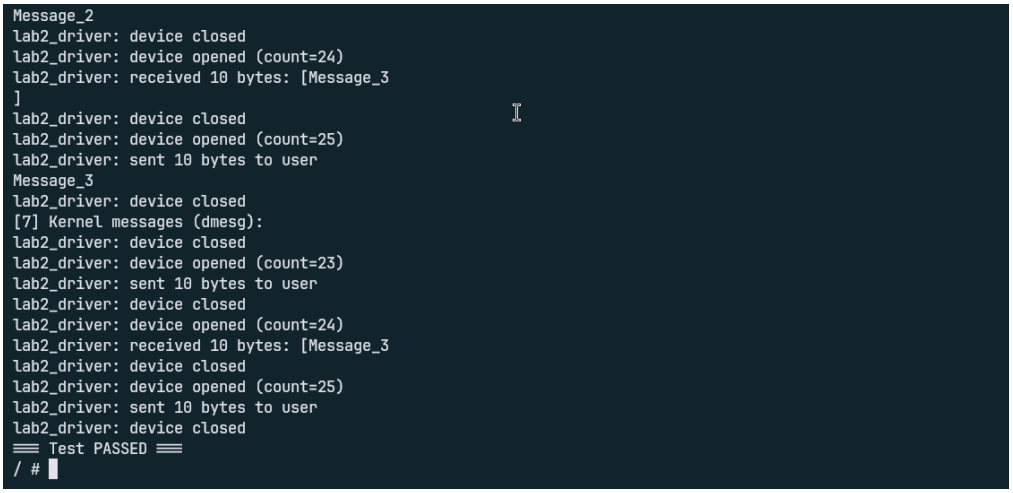


Figure 4: `test_driver.sh` (continued): three write/read rounds and `dmesg`, ending with `Test PASSED`

Every write produces the log sequence `device opened` → `received N bytes` → `device closed`, and the read that follows returns exactly the string just written (`Read back: [Hello from userspace, LAB-02!]`). The string is 30 bytes because `echo` appends a newline, which is also why the closing `]` is pushed onto the next line in the log. In the first screenshot the first run stops after `[4]`; running it again completes the whole script. `open_count` keeps growing across runs because the driver increments it in `open` but never decrements it in `release`, so it counts the total number of opens rather than the number of currently open files.



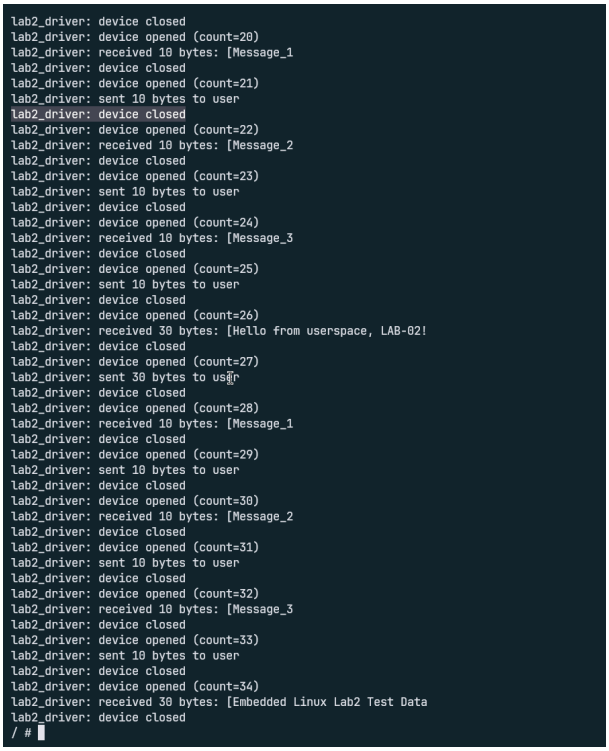


Figure 5: `dmesg | grep lab2`: every operation since the module was loaded

Part B – procfs and sysfs

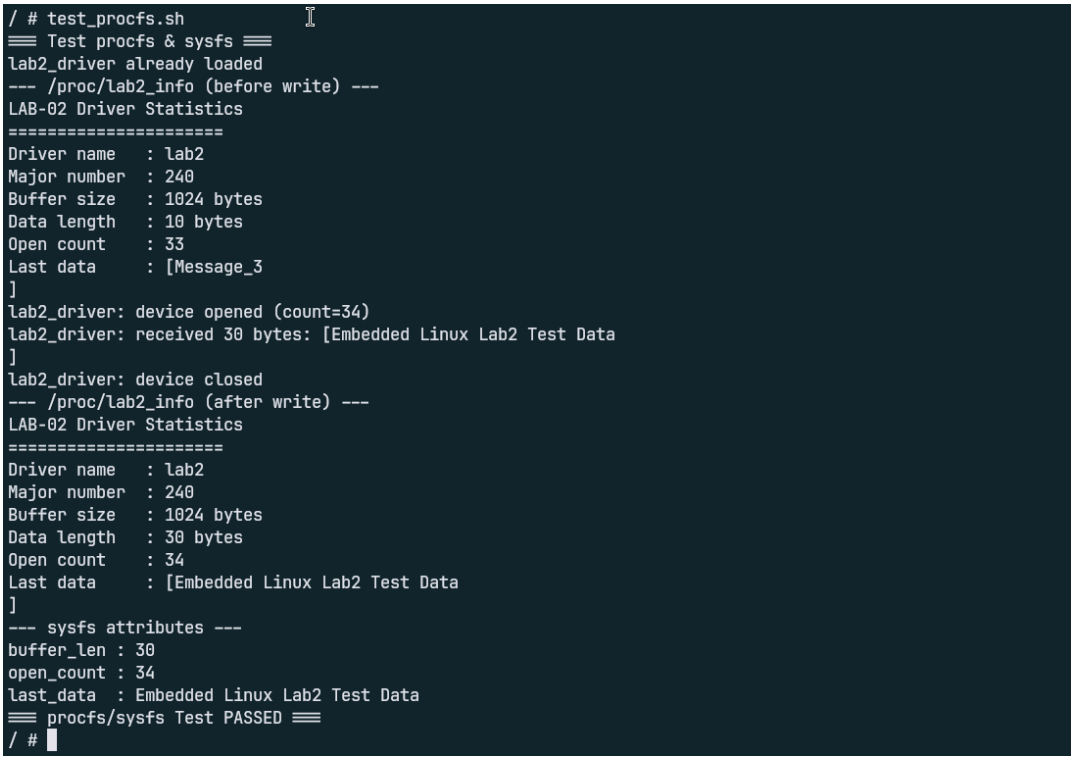


Figure 6: `test_procfs.sh`: `/proc/lab2_info` before and after a write, and the sysfs attributes

Before the write, `/proc/lab2_info` still holds the data of the last write from the previous test (`Message_3`, 10 bytes, open count 33). After writing `Embedded Linux Lab2 Test Data`, the data length becomes 30 bytes and the open count 34. The three sysfs files report the same values.

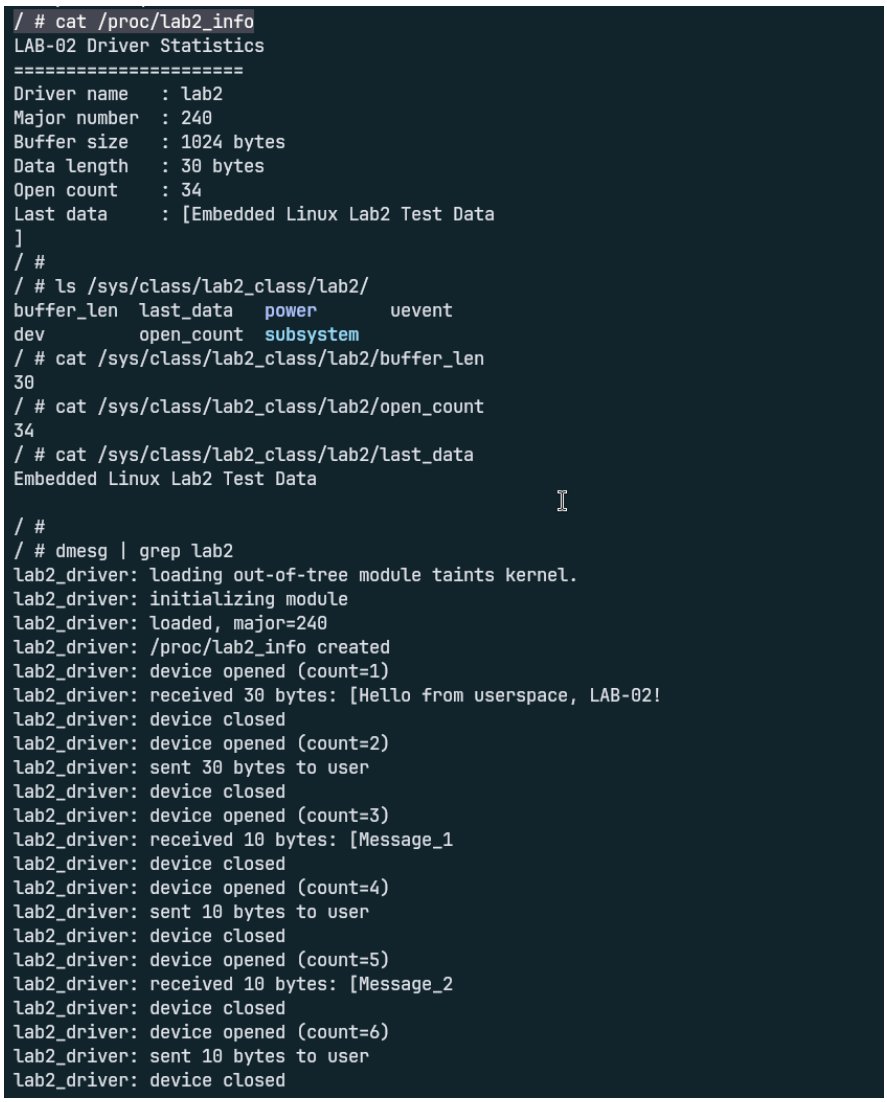


Figure 7: Reading `/proc/lab2_info` on its own, listing and reading each attribute in `/sys/class/lab2_class/lab2/`

Besides the driver's three attributes (`buffer_len`, `open_count`, `last_data`), the sysfs directory also contains `dev`, `uevent`, `power` and `subsystem`, which the driver core creates for every device. `dev` contains `240:0`; this is what udev/mdev reads to create the `/dev/lab2` node automatically.

Part C – MTD and JFFS2 (on the host)



Figure 8: `mtdinfo /dev/mtd1` and `/proc/mtd`

After `modprobe nandsim`, `/proc/mtd` lists two devices. `mtd0`, named `"BIOS"`, is not the simulated flash but the SPI flash chip holding the laptop's real firmware, exposed by a host driver. The simulated NAND is `mtd1`: 32 MiB, 16 KiB erase blocks (2048 blocks), 512-byte pages, 16-byte OOB. For this reason every command in the handout that uses `mtd0` was changed to `mtd1` (see section 6).

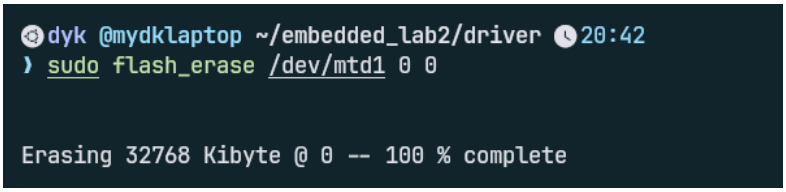


Figure 9: `flash_erase /dev/mtd1 0 0`: erasing all 32 MiB

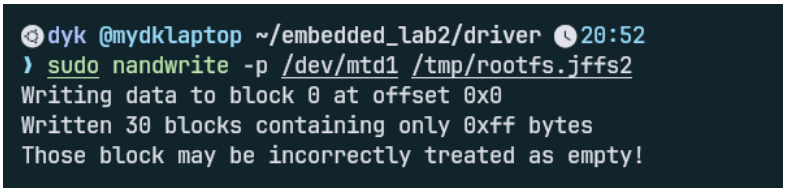


Figure 10: `nandwrite -p /dev/mtd1 /tmp/rootfs.jffs2`

The JFFS2 image was created with `mkfs.jffs2 --eraseblock=0x4000`, matching nandsim's 16 KiB erase block. The message "Written 30 blocks containing only 0xff bytes" comes from `--pad`, which fills the image with `0xFF`; for JFFS2, regions full of `0xFF` are simply erased free space, so this is harmless.

```
dyk @mydklaptop ~/embedded_lab2/driver 20:53
❯ sudo mkdir -p /mnt/jffs2
sudo mount -t jffs2 /dev/mtdblock1 /mnt/jffs2

ls -la /mnt/jffs2/
cat /mnt/jffs2/hello.txt
cat /mnt/jffs2/info.txt
cat /mnt/jffs2/logs/boot.log
total 4
drwxr-xr-x 4 root root    0 Jan  1  1970 .
drwxr-xr-x 3 root root 4096 Sep 28 20:56 ..
-rw-rw-r-- 1 dyk  dyk    18 Sep 28 20:52 hello.txt
-rw-rw-r-- 1 dyk  dyk    20 Sep 28 20:52 info.txt
drwxrwxr-x 2 dyk  dyk     0 Sep 28 20:52 logs
Hello from JFFS2!
Embedded Linux Lab2
Mon Sep 28 08:52:30 PM +07 2026
```

Figure 11: Mounting JFFS2 through `/dev/mtdblock1` and reading the files

```
dyk @mydklaptop ~/embedded_lab2/driver 21:01
❯ sudo nanddump -o -l 512 -f /tmp/dump.bin /dev/mtd1
hexdump -C /tmp/dump.bin | head -20
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 16384, page size 512, OOB size 16
Dumping data starting at 0x00000000 and ending at 0x00000200...
00000000  85 19 01 e0 31 00 00 00  1d d9 82 42 01 00 00 00  |....1......B....|
00000010  00 00 00 00 02 00 00 00  1e 71 ba 6a 09 08 00 00  |.........q.j....|
00000020  92 1c ba b1 15 da 69 f4  68 65 6c 6c 6f 2e 74 78  |......i.hello.tx|
00000030  74 ff ff ff 85 19 02 e0  56 00 00 00 09 64 e7 62  |t.......V....d.b|
00000040  02 00 00 00 01 00 00 00  b4 81 00 00 e8 03 e8 03  |................|
00000050  12 00 00 00 1e 71 ba 6a  1e 71 ba 6a 1e 71 ba 6a  |.....q.j.q.j.q.j|
00000060  00 00 00 00 12 00 00 00  12 00 00 00 00 00 00 00  |................|
00000070  ee dd 4b b3 d2 39 36 f9  48 65 6c 6c 6f 20 66 72  |..K..96.Hello fr|
00000080  6f 6d 20 4a 46 46 53 32  21 0a ff ff 85 19 01 e0  |om JFFS2!.......|
00000090  30 00 00 00 78 be 3e fa  01 00 00 00 01 00 00 00  |0...x.>.........|
000000a0  03 00 00 00 1e 71 ba 6a  08 08 00 00 09 a6 cb 3c  |.....q.j.......<|
000000b0  a9 de 79 72 69 6e 66 6f  2e 74 78 74 85 19 02 e0  |..yrinfo.txt....|
000000c0  58 00 00 00 3a 13 38 82  03 00 00 00 01 00 00 00  |X...:.8.........|
000000d0  b4 81 00 00 e8 03 e8 03  14 00 00 00 1e 71 ba 6a  |.............q.j|
000000e0  1e 71 ba 6a 1e 71 ba 6a  00 00 00 00 14 00 00 00  |.q.j.q.j........|
000000f0  14 00 00 00 00 00 00 00  8a 74 43 dd 56 6e 9a a4  |.........tC.Vn..|
00000100  45 6d 62 65 64 64 65 64  20 4c 69 6e 75 78 20 4c  |Embedded Linux L|
00000110  61 62 32 0a 85 19 01 e0  2c 00 00 00 5f 56 f1 e0  |ab2.....,..._V..|
00000120  01 00 00 00 02 00 00 00  04 00 00 00 ae 70 ba 6a  |.............p.j|
00000130  04 04 00 00 bb 60 d6 52  40 19 cb d1 6c 6f 67 73  |.....`.R@...logs|

dyk @mydklaptop ~/embedded_lab2/driver 21:04
❯
```

Figure 12: `nanddump` of the first 512 bytes of `mtd1`

The raw dump shows that JFFS2 stores data as a sequence of nodes. Each node starts with the magic `85 19` (`0x1985` in little-endian) followed by the node type: `01 e0` is a directory entry (dirent) node carrying the file name `hello.txt`, and `02 e0` is an inode node carrying the content `Hello from JFFS2!`. Similar node pairs follow for `info.txt` and the `logs` directory. There is no FAT and no fixed superblock; at mount time JFFS2 scans these nodes to rebuild the directory tree.

Part D – Automatic module loading by BusyBox init

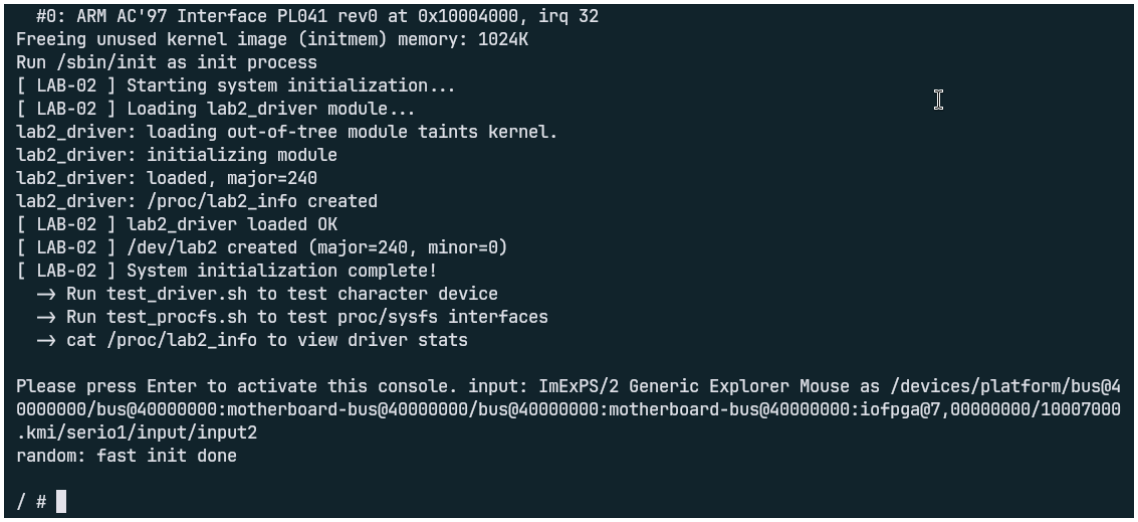


Figure 13: Boot log: rcS loads `lab2_driver` and creates `/dev/lab2`

`/etc/inittab` contains `::sysinit:/etc/init.d/rcS`, so rcS runs before the shell starts. rcS mounts `proc`, `sysfs`, `tmpfs` and `devtmpfs`, checks that `/lib/modules/5.15.0/lab2_driver.ko` exists and then runs `insmod`. The log shows the whole sequence: the kernel reports that the out-of-tree module taints the kernel (normal for any module built outside the kernel tree), the driver prints `initializing`, `loaded, major=240` and `/proc/lab2_info created`, and rcS then creates `/dev/lab2` and prints `System initialization complete!`. The complete output of `test_driver.sh` and `test_procfs.sh` on this booted system is shown in the figures of parts A and B.

Because rcS has already loaded the module, the original `test_driver.sh` would call `insmod` again and fail with `File exists`. We changed both scripts to check `lsmod | grep -q '^lab2_driver'` first and skip loading if the module is already there (the line `lab2_driver already loaded` in the log).

4. File system comparison

Criterion	ext4	JFFS2	cramfs	UBIFS (on UBI)
Writable	Yes	Yes	No (read-only)	Yes
Wear leveling	None of its own; relies on the eMMC/SD controller	Yes, log-structured, within one partition	Not needed, never written	Yes, done by UBI across the whole flash
Compression	No	Yes (zlib, lzo, rtime)	Yes (zlib, per page)	Yes (lzo, zlib, zstd)
Mount/boot time	Fast, reads the superblock	Slow, scans every node; grows with size	Fast	Fast, on-flash index
Power-loss tolerance	Journal	Good, CRC on every node	Not applicable	Good, journal + CRC
Suitable devices	eMMC, SD, hard disks (block devices)	Small raw NAND/NOR	Read-only firmware, ROM	Medium and large raw NAND

Recommendation for an IoT device with 32 MB NAND and OTA updates. We would use UBI as the flash management layer, split into two rootfs volumes A/B using SquashFS (read-only, good compression) and a small data volume using UBIFS. During an OTA update the device writes the new image into the volume that is not running and then changes a U-Boot environment variable so that the next boot uses it; if that boot fails, it falls back to the old volume. UBI handles wear leveling and bad blocks across the whole chip, so both the frequent writes (logs, configuration) and the large rewrites during OTA are spread evenly. JFFS2 would still work at 32 MB and is simpler, but its mount time grows with the size of the flash and it does not support safely replacing the whole rootfs the way the A/B scheme above does. ext4 cannot be used at all, because raw NAND is not a block device.

5. Answers to the analysis questions

Question 1. Why not use memcpy() between the kernel buffer and the user buffer

Every process has its own virtual address space. The `buf` pointer that `lab2_read()` receives is a virtual address of the calling process (for example `cat`). It only has meaning through that process's page tables, and since it comes from user space it cannot be trusted: it may be `NULL`, point to an unmapped region, point to a page that is swapped out, or deliberately point into kernel memory.

`memcpy()` checks none of this. If the page is not in RAM, a page fault occurs inside the kernel with nothing to handle it, which becomes a kernel oops. If the user passes a kernel address, the driver would read or overwrite kernel memory on the user's behalf, which is a security hole.

`copy_to_user()`/`copy_from_user()` do three things `memcpy()` does not: they check that the whole range `[buf, buf+len)` lies in the user part of the address space (`access_ok`); they copy with instructions registered in the exception table, so a page fault either brings the page in and continues or stops and returns the number of bytes not copied instead of oopsing; and on ARM they also enable and disable access to the user region (domains) at the right moment. In our driver a non-zero return value is turned into `-EFAULT`, so a `read()` with a bad pointer only makes that program get an error; the kernel is unaffected.

Question 2. Why a mutex is needed

`device_buffer` and `buffer_len` are global variables shared by every process that opens `/dev/lab2`. On an SMP kernel like ours (4 CPUs), two processes can be inside `lab2_write()` at the same time on different CPUs, and even on one CPU `copy_from_user()` may sleep on a page fault and let another process run in between.

Without the mutex: process A writes "AAAAAAAAAA" (10 bytes) while process B writes "BBB" (3 bytes). A copies its 10 bytes, B overwrites the first 3, and A then sets `buffer_len = 10` after B. Reading returns "BBBAAAAAAA", a string nobody wrote. `open_count++` is not atomic either: it is a read–add–write, and two CPUs doing it at once can lose a count.

The mutex guarantees that only one process is inside the copy-and-update-length section at a time. We use a mutex rather than a spinlock because `copy_from_user()` may sleep, and sleeping while holding a spinlock is not allowed.

Re-reading the code, we noticed that `lab2_read()` and the sysfs `*_show()` functions read `device_buffer` without taking the mutex, so a read that overlaps a write could still return half old and half new data. To be strict, the read path should lock as well. In our tests reads and writes happened one after another, so this case did not occur.

Question 3. procfs vs sysfs

procfs was originally meant for information about processes (`/proc/<PID>/`), but over time drivers put all kinds of information there, each file in its own format with no common structure. sysfs arrived with the driver model: every directory is a kobject (device, driver, bus, class), and the rule is *one value per file*. Programs and udev can read it without parsing text, and the directory tree reflects the real relationships between devices. This is why the kernel documentation advises against adding new non-process files to `/proc`.

Our driver has both, and the difference is clear: `/proc/lab2_info` prints a labelled multi-line table that is easy for a human to read, written with `seq_file` and `single_open()` (the kernel handles offsets and paging; the driver only calls `seq_printf`). `/sys/class/lab2_class/lab2/` has three files, `buffer_len`, `open_count` and `last_data`, one value each, created with `DEVICE_ATTR_RO` and `sysfs_create_group()` on the device's kobject. A script that needs the buffer length just runs `cat buffer_len`, whereas with procfs it has to `grep` and cut the string. For a real driver we would keep only the sysfs part (or debugfs for debug information).

Question 4. Why JFFS2 must erase a whole block before writing

In NAND flash every bit is a cell with a floating gate. Programming can only pull a bit from 1 to 0, by injecting charge into the gate. Returning a bit to 1 requires removing that charge, and the chip can only do this for a large group of cells at once, called an erase block. Writes happen per page (512 bytes on our nandsim) while erases happen per block (16 KiB, i.e. 32 pages). After an erase every byte reads `0xFF`; that is why `flash_erase` in part C set the whole chip to `0xFF`, and why `nandwrite` treats all-`0xFF` blocks as empty.

So a few bytes cannot be overwritten in place. JFFS2 solves this by never overwriting: every change is a new node (with a higher version number) written to the next free space, and the old node is marked obsolete. When space runs out, the garbage collector picks a block, copies its still-valid nodes elsewhere and erases the block. Writing sequentially like this also spreads the erase count evenly over the blocks.

ext4 works on a block device, where each sector can be overwritten directly. On eMMC or SD the controller inside the card (the FTL) already hides erasing and wear leveling, so ext4 never needs to know it is on flash. Put ext4 on raw NAND and nobody does that work.

Question 5. What happens if insmod fails in rcS

The system keeps booting. rcS is an ordinary shell script without `set -e`, so a failing command just returns a non-zero code and the script carries on with the next lines. BusyBox init does not care whether rcS succeeded either: once the `sysinit` entry finishes it moves on to `askfirst` and opens the shell. The user still gets a shell, but there is no `/dev/lab2`, and anything that depends on the driver will fail later.

Our rcS already handles two cases: if the `.ko` file does not exist it prints `WARNING: lab2_driver.ko not found`, and if `insmod` returns an error it prints `ERROR: insmod lab2_driver failed!` and does not create the node. To make this more robust, the error could be written to a log file on a persistent partition, loading could be retried a few times, or, for a mandatory driver, rcS could reboot into a backup firmware image (combined with the A/B scheme in section 4).

6. Remarks and difficulties

Difficulty 1: `mtd0` on our machine is the real BIOS flash

The handout assumes that after `modprobe nandsim` the simulated NAND is `/dev/mtd0`, and the `flash_erase`, `nandwrite` and `nanddump` commands all target it. On our laptop, `cat /proc/mtd` showed `mtd0` named `"BIOS"`, 32 MiB in size with a 4 KiB erase size: this is the SPI chip holding the machine's firmware, which the host kernel exposes as an MTD device. Running `flash_erase /dev/mtd0 0 0` as written would have wiped the firmware and left the machine unable to boot. We checked the names in `/proc/mtd` first, identified the simulated NAND as `mtd1` ("NAND simulator partition 0", 16 KiB erase size), and switched every command in part C to `mtd1`/`mtdblock1`. The lesson is to always check a device's name and size before running an erase command on it, rather than trusting its index.

Difficulty 2: insmod twice

After adding part D, rcS loads the module at boot, so the original `test_driver.sh` and `test_procfs.sh` called `insmod` again and failed with `File exists`. We changed both scripts to check `lsmod` first and to run `mknod` only when the node does not exist yet.

Remarks

This lab showed us that a driver is essentially a table of function pointers that VFS calls into, and that everything else (creating the node, exporting information, loading at boot) revolves around registering that table correctly. The MTD part showed how different raw flash is from a disk: only by looking at the hexdump did we see that JFFS2 really is just a chain of nodes, with no fixed structure to overwrite.